

Assignment 2

Sanskrit-to-English Neural Machine Translation

A custom BiLSTM Seq2Seq model with additive attention

Student Information

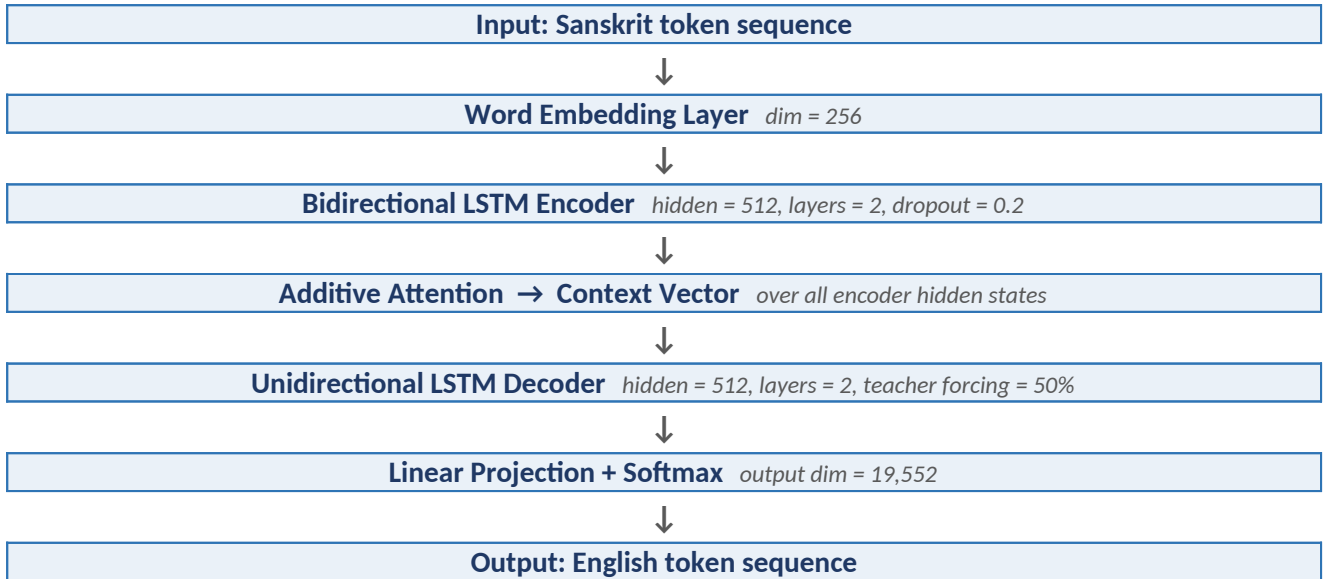
- Name: Ayan Sengupta
- Roll Number: G25AIT1042
- Project Repository: <https://github.com/Ayan47/Sanskrit-to-English-Neural-Machine-Translation>

1. Introduction

This report details the development, training, and evaluation of a Neural Machine Translation (NMT) system engineered to translate text from Sanskrit into English. Translating Sanskrit is a demanding task: the language is morphologically rich, applies complex sandhi (word-fusion) rules, and is comparatively low-resource. To address these challenges, a custom Sequence-to-Sequence (Seq2Seq) architecture built on a Bidirectional LSTM (BiLSTM) encoder and an additive attention mechanism was implemented entirely from scratch. The system complies strictly with the assignment constraints: no external translation APIs and no pre-trained machine-translation models were used at any stage.

2. System Architecture & Diagram

The translation model follows a Sequence-to-Sequence framework augmented with attention, allowing the decoder to focus on the most relevant source positions at each generation step. The end-to-end data flow through the network layers is shown below.



2.1 Encoder & Decoder Mechanics

The Encoder processes the source Sanskrit text bidirectionally, capturing contextual dependencies from both preceding and following tokens. It maps the 33,276-token source vocabulary into a dense 256-dimensional embedding space, then passes the sequence through two stacked LSTM layers with a hidden size of 512 per direction.

The Decoder consumes the (direction-summed) final encoder state together with a context vector produced by the additive attention module at each time step. It generates predictions token by token over the 19,552-token English vocabulary. During training, teacher forcing is applied with probability 0.5 so the decoder learns to recover from its own errors; at inference the model decodes autoregressively.

3. Training Procedure & Hyperparameters

3.1 Tokenization and Vocabulary Generation

The dataset was processed with whitespace tokenization. To keep the model within the provided data distribution, the Sanskrit and English vocabularies were built strictly from the training split (train_sa and train_en), yielding 33,276 source tokens and 19,552 target tokens at a minimum frequency of 1. Four structural tokens (<PAD>, <UNK>, <SOS>, <EOS>) were injected into the pipeline, and maximum sequence lengths were derived dynamically from the training set.

3.2 Hyperparameter Configuration

Hyperparameter	Value	Rationale
Batch Size	64	Balances memory use with gradient stability.
Learning Rate	0.001	Standard initialization for the Adam optimizer.
Teacher Forcing Ratio	0.5	Lets the decoder learn from its own predictions half the time, improving inference robustness.
Dropout	0.2	Regularization to reduce node co-adaptation and overfitting.
Embedding Dim	256	Dense representation for both source and target tokens.
Hidden Dim / Layers	512 / 2	Encoder and decoder capacity.

4. Experimental Results & Loss Analysis

The model comprises 59,591,776 trainable parameters. It was trained for up to 15 epochs with an early-stopping strategy driven by the validation loss; the best checkpoint (Epoch 5) was preserved and used for all evaluation reported here. The run analysed in this report loaded that checkpoint directly and performed evaluation only (no retraining).

4.1 Training vs. Validation Loss History

Epoch	Training Loss	Validation Loss	Model Status
1	7.2781	7.0442	Saved
2	6.7293	7.0554	—
3	6.4060	6.9648	Saved
4	6.1028	6.9051	Saved
5	5.8177	6.8804	BEST MODEL SAVED
6	5.5255	6.9274	Overfitting begins
...	...	...	—
15	2.9850	7.7643	Severe overfitting

4.2 Discussion on Overfitting

The model reached its generalization peak at Epoch 5. Beyond that point the network increasingly memorized the training data: training loss fell steeply (bottoming out near 2.9850 by Epoch 15) while validation loss worsened markedly (rising to 7.7643). The checkpoint-tracking mechanism mitigated this by preserving the Epoch-5 weights (validation loss 6.8804) for final evaluation, rather than the overfit late-epoch weights.

4.3 Quantitative Evaluation Metrics

Evaluation was performed with batched greedy decoding for the metric computation (roughly 0.85 s per 1,000 sentences on GPU), while the submission file and qualitative examples were generated with beam search (width 5, length penalty 0.7). Two metrics were computed: corpus BLEU (NLTK) and a corpus-level, micro-averaged unigram token overlap score reported as precision, recall, and F1. The results on the dev and test splits (1,000 sentences each) are:

Split	BLEU	Token Precision	Token Recall	Token F1
Dev	0.0128	0.1828	0.1255	0.1489
Test	0.0107	0.1775	0.1242	0.1461

The BLEU scores are very low (~ 0.01) and the token-level F1 scores are modest (~ 0.15). Taken together with the loss history, these figures indicate that although the best checkpoint was selected at the point of minimum validation loss, the model's absolute translation quality remains weak — consistent with the high validation loss (6.88) even at its best epoch and with the difficulty of the low-resource, morphologically dense source language.

5. Qualitative Translation Examples

The examples below are drawn directly from the evaluation run and generated with beam search (width 5, length penalty 0.7). They illustrate both the partial successes and the characteristic failure modes of the model.

Sanskrit: ते वीराः ।

Reference: *Those are brave men.*

Prediction: You are the

Sanskrit: यूयं भक्तियोगं पठथ ।

Reference: *You all are studying bhakti yoga.*

Prediction: You all the

Sanskrit: पाठे'स्मिन् वयम्, awk ... if, else, else if ...

Reference: *In this tutorial, we will learn about if, else, else if in awk.*

Prediction: In this tutorial, we will learn how how to the in this

Sanskrit: 'इन्फ़ैनेट लूप' ... निरुत्तरां कारयति ।

Reference: *Infinite loop can cause the system to become unresponsive.*

Prediction: Click on the the and the the

5.1 Error Analysis

The model captures some surface structure of the target language: it reproduces common English function words and recognizable templates such as “You are...” and “In this tutorial, we will learn...”, and it handles embedded English-loan words and punctuation reasonably well. However, the dominant failure mode is degenerate repetition (“the the the...”) and a general inability to recover source content words, which is why

BLEU is near zero despite outputs that look superficially English. Additional errors arise from out-of-vocabulary tokens collapsing to <UNK> due to vocabulary sparsity (minimum frequency = 1), and from the 1-to-1 whitespace tokenization being unable to resolve Sanskrit sandhi.

6. Conclusion and Future Improvements

The custom Seq2Seq BiLSTM with additive attention successfully learns basic translation alignments and target-side fluency patterns, but its absolute accuracy on this low-resource, morphologically dense language pair is limited, as reflected in the low BLEU and modest token-F1 scores. The word-level tokenization scheme, in particular, cannot decompose Sanskrit sandhi, inflating the source vocabulary and diluting learned representations. Priorities for future work include: adopting subword tokenization (e.g. Byte-Pair Encoding) to shrink the source vocabulary and densify representations; adding a repetition penalty or coverage mechanism to suppress the degenerate “the the” outputs; increasing training data or applying stronger regularization and early stopping to combat the overfitting evident after Epoch 5; and experimenting with scheduled sampling in place of a fixed teacher-forcing ratio.

7. References and Disclosures

- Pre-trained models disclosed: The core machine-translation model is entirely custom and coded from scratch using the torch.nn library. No pre-trained translation models were used.
- Evaluation metrics: Corpus BLEU was computed with NLTK. A corpus-level micro-averaged unigram token-overlap score (precision / recall / F1) was computed directly from the model outputs. BERTScore was not used in this run.
- Libraries used: PyTorch (neural network implementation), NLTK (BLEU and tokenization support), pandas / NumPy (data handling), tqdm (progress reporting).
- Source code repository: <https://github.com/Ayan47/Sanskrit-to-English-Neural-Machine-Translation>